Map Tiles Stress Test — Martin vs Tileserver

GKE Autopilot · me-central2 · dev · sat-style · 1 pod / 2 vCPU / 4 GB each · k6 constant-VUs, 60s

martin = Rust Martin 1.11.0 · 8 workers | tileserver = Node tileserver-gl v5.6 · 1 process

Verdict: Under high concurrency, **Martin wins decisively**. On the same per-pod resources, Martin served **~2,862 req/s** at sub-second p95 with ~0% errors at 5,000 VUs, while Tileserver **collapsed at 99.9%** failures — a **~24×** difference in sustained throughput. Root cause: Rust async + 8 worker processes (multi-core) vs a single Node event loop (one core). Matching one Martin pod would take **~24 Tileserver pods**.

0.42%
Martin — error rate @ 5,000 VUs

99.91%
Tileserver — error rate @ 5,000 VUs

1 · Test Configuration

BACKEND PODS — IDENTICAL FOR BOTH (SAME BOX)

Parameter	Value
Replicas	1 (HPA disabled)
App CPU (request = limit)	2000m (2 vCPU)
App memory (request = limit)	4 GiB
QoS class	Guaranteed
gcsfuse sidecar CPU	1000m
gcsfuse sidecar memory	2 GiB
gcsfuse ephemeral storage	5 GiB
gcsfuse file cache	2 GiB (LRU bounded)
gcsfuse metadata cache	ttl ∞ · parallel downloads on
gcsfuse max-conns-per-host	0 (unlimited)
Istio sidecars	istio-init + istio-proxy · 1 GiB ephemeral each
Pod ephemeral budget	7 GiB (Autopilot cap 10 GiB / pod)
Scheduled scaler	night scale-to-0 disabled

ENGINE-SPECIFIC

Parameter	martin	tileserver
Image	Rust Martin 1.11.0	Node tileserver-gl v5.6
Concurrency model	8 worker processes (async)	1 process / 1 event loop
In-engine cache	cache_size_mb = 1024	—
Node heap	—	--max-old-space-size=3072
Container port	3000	8080
Tile sources	PostGIS (topology) + gcsfuse mbtiles/pmtiles	gcsfuse mbtiles/pmtiles
Database	PostgreSQL 16.13 / PostGIS 3.6 — direct private IP 10.245.0.3	none
Database machine	Cloud SQL Enterprise Plus · 2 vCPU / 16 GB	n/a

K6 LOAD GENERATOR

Parameter	Value
Machine	GCE n2-custom-8-16384 (8 vCPU / 16 GB)
Image	Debian 12
Placement	in-VPC (na-dev-vpc), me-central2, no external IP
Executor	constant-vus · sleep(1) per iteration · 60s
Workload	HAR-captured sat-style tile URLs — ~7,700 (tileserver) / ~9,900 (martin) unique

2 · Head-to-head @ 5,000 VUs



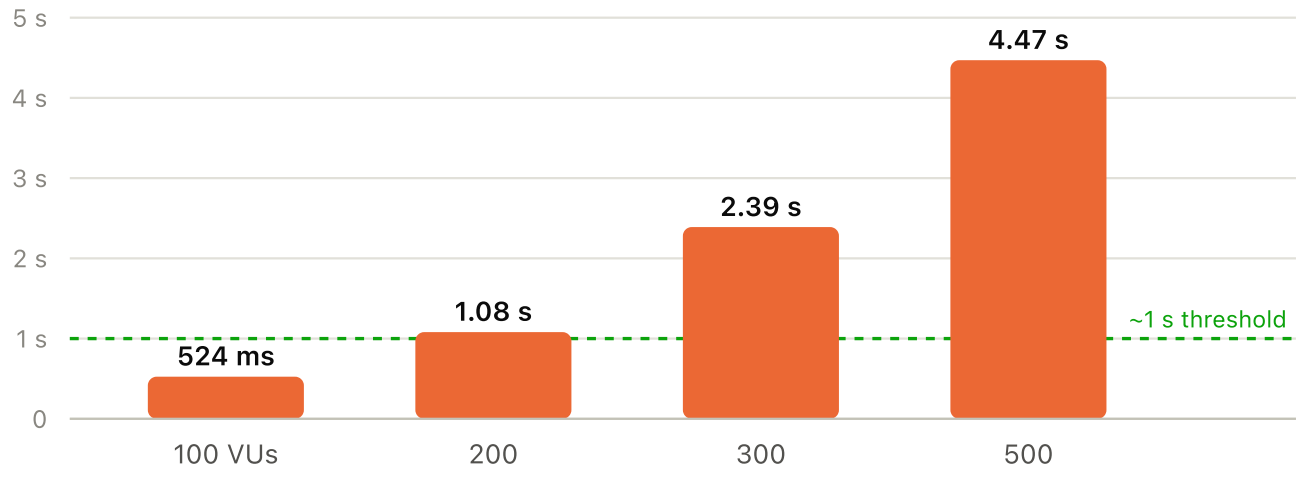
Request error rate. Tileserver rejected nearly all connections at 5,000 VUs.

Metric @ 5,000 VUs	tileserver	martin
Error rate	99.91%	0.42%
p95 (successful requests)	18.4 s	825 ms
median (successful)	3.47 s	298 ms
Successful req/s	~2	~2,862
Tile data actually served	33 MB	1.2 GB

Reading caveat: the blended `http_req_duration` p95 for Tileserver reads ~49 ms, which is **misleading** — 99.9% of requests failed instantly (connection refused in ms), pulling the blended figure down. The real number is the `{expected_response:true}` sub-metric read together with the error rate.

3 · Load curve — Tileserver (1 pod, warm cache)

Tileserver stays healthy (0% errors) up to 500 VUs but latency climbs fast; throughput plateaus at a **~120 req/s** ceiling and does not rise with more load — the knee sits around **200 VUs** (p95 ~1 s).



p95 of successful requests. At **5,000 VUs** Tileserver collapses entirely (99.9% errors) — not plotted here.

VUs	Errors	p95 (success)	median	req/s
100	0%	524 ms	374 ms	73
200	0%	1.08 s	894 ms	105
300	0%	2.39 s	1.94 s	114
500	0%	4.47 s	3.60 s	120
5000	99.91%	(18.4 s)	—	~2 ✓

4 · Martin (1 pod, warm cache)

VUs	Errors	p95 (success)	median	req/s
5000	0.42%	825 ms	298 ms	2,862

At 5,000 VUs Martin is still sub-second with ~0% errors — it has **not reached its knee**. It was not pushed higher because the 8-vCPU load-generator VM would saturate before Martin does.

5 · Why the gap

- **martin** — Rust + async (tokio), `worker_processes: 8` → genuinely uses multiple cores and holds thousands of concurrent connections cheaply; in-RAM tile cache.
- **tileserver-gl** — Node, **1 process / 1 event loop → capped at ~1 core**. No cluster mode (confirmed entrypoint `node main.js`). At 500 concurrent it already queues into multi-second latency; at 5,000 the listener cannot accept connections fast enough and rejects almost all of them.
- The sat-style workload is ~90% static tiles from gcsfuse (only ~10% touches PostGIS on Martin). Both are gcsfuse-bound for tiles, so the difference comes from **connection handling + multi-core**, not tile read speed.

Source: k6 constant-VUs, 60s, from GCE `n2-custom-8-16384` in-VPC (na-dev-vpc, me-central2). URLs replayed from HAR auto-capture (Playwright, sat-style): ~7,700 tileserver / ~9,900 martin unique tiles. The client was not the bottleneck (no `dropped_iterations`; full VU count reached). Report generated 2026-07-15.